LLM4DyG Figure 3 Reproduction Report: Vicuna-7B vs GPT-3.5

Paper: *LLM4DyG: Can Large Language Models Solve Spatial-Temporal Problems on Dynamic Graphs?* (KDD 2024)
Authors: Zeyang Zhang, Xin Wang, Ziwei Zhang, Haoyang Li, Yijian Qin, Wenwu Zhu
Reproduction Date: March 2026
Reproduction Model: Vicuna-7B (locally hosted)
Original Model (Paper): GPT-3.5-Turbo (OpenAI API)

---

1. What the Original Paper Did

1.1 Motivation

LLM4DyG is the first systematic benchmark for evaluating Large Language Models on **spatial-temporal reasoning over dynamic graphs**. Dynamic graphs — where edges appear at specific timestamps — are ubiquitous in real-world networks (social networks, communication logs, financial transactions). The paper asks: *Can LLMs understand temporal evolution and spatial structure of dynamic graphs?*

1.2 Benchmark Design

The authors designed **9 tasks** spanning temporal and spatial reasoning dimensions:

#	Task	Type	Question	Answer Format
1	`when_link`	Temporal	When are two nodes directly linked?	List of timestamps
2	`when_connect`	Temporal	When are two nodes first connected (via any path)?	Single integer
3	`when_tclosure`	Temporal	When do three nodes form a closed triad?	Single integer
4	`what_node`	Spatial	What nodes are neighbors of a node at time t?	List of node IDs
5	`which_neighbor`	Spatial	Which nodes become neighbors only after time t?	List of node IDs
6	`check_tclosure`	Verification	Do three nodes form a triangle?	Yes/No
7	`check_tpath`	Verification	Is a given path time-respecting?	Yes/No
8	`find_tpath`	Generation	Find a chronological path of length ≥ 3	List of node IDs
9	`sort_edge`	Ordering	Sort all edges by timestamp	List of tuples

1.3 Figure 3 — The Core Evaluation

Figure 3 presents a **3×3 heatmap grid** for each of the 9 tasks, varying:

- **T** ∈ {10, 20, 30} — number of time steps (rows)
- **p** ∈ {0.3, 0.5, 0.7} — edge probability in Erdős-Rényi generation (columns)
- **N = 10** nodes (fixed)
- **100 instances** per (T, p) configuration
- **Accuracy (%)** displayed in each cell

The original Figure 3 used **GPT-3.5-Turbo** with 2-shot prompting, no chain-of-thought, and reported accuracy percentages showing how performance varies across task complexity and graph structure.

---

2. What We Changed

2.1 Model Substitution

Aspect	Original (Paper)	Our Reproduction
Model	GPT-3.5-Turbo	Vicuna-7B
Parameters	~175B (estimated)	7B
Access	OpenAI API (cloud)	Locally hosted via FastChat
Cost	API charges per token	Free (GPU compute only)
Context window	4K–16K tokens	~2K tokens
Training data	Proprietary	Open (LLaMA + ShareGPT)

2.2 Infrastructure

- **GPU:** Local GPU via NVIDIA CUDA
- **Server:** FastChat OpenAI-compatible API server (controller + model worker + API server)
- **Framework:** Identical LLM4DyG codebase — same data generation, same prompting, same evaluation

2.3 What Remained Identical

- All 9 tasks, same task implementations
- Same graph generator (`DyGraphGenERCon` — Erdős-Rényi with continuous timestamps)
- Same parameters: N=10, T∈{10,20,30}, p∈{0.3,0.5,0.7}
- Same 100 instances per configuration (num_seed=100)
- Same 2-shot prompting (k=2)
- Same evaluation/parsing logic
- No chain-of-thought (add_cot=0), no role instruction (add_role=0)

---

3. How We Reproduced It

3.1 Execution Strategy

The 9 tasks × 9 configurations = **81 experiment groups** (8,100 total LLM queries) were executed using a GPU-safe sequential strategy:

1. **Phase 1 — Data Generation:** All 81 configurations generated in parallel (CPU-only, fast)
2. **Phase 2 — Model Inference:** Tasks run sequentially (one task at a time) to prevent GPU OOM/thrashing on the shared GPU
3. **Phase 3 — Evaluation:** All results parsed and scored in parallel (CPU-only)

3.2 Execution Scripts

- First 4 tasks (`when_link`, `when_connect`, `when_tclosure`, `what_node`) run via `run_fig3_parallel_safe.sh` → results in `logs/fig3/`
- Remaining 5 tasks (`which_neighbor`, `check_tclosure`, `check_tpath`, `find_tpath`, `sort_edge`) run separately → results in `logs/fig3_remaining/`

3.3 Per-Instance Pipeline

For each of the 100 instances per (task, T, p):

```
Generate ER graph → Create Q&A → Format 2-shot prompt → Query Vicuna-7B →
Parse response → Score accuracy
```

3.4 Sample Prompts and Model Responses

Below is one actual prompt–response pair per task (all from T=10, p=0.3, seed 0).

Task 1: `when_link`

Prompt:

> In an undirected dynamic graph, (u, v, t) means that node u and node v are linked with an undirected edge at time t. Your task is to find ALL time steps at which two specified nodes share a direct edge in the dynamic graph.
>
> *Here are 2 examples:* Question: Given an undirected dynamic graph with the edges [(0, 2, 0), (0, 3, 1), (1, 2, 5), (3, 1, 6)]. When are node 0 and node 3 linked? ... Answer:[1]
>
> Question: Given an undirected dynamic graph with the edges [(1, 5, 0), (1, 8, 3), (3, 5, 3), (6, 8, 3), (0, 4, 5), (7, 9, 5), (4, 7, 7), (5, 6, 9)]. **When are node 3 and node 5 linked?**

Ground Truth: `[3]`

Vicuna-7B Response (530 tokens):

> Answer: [3, 5]

Verdict: ❌ Incorrect — the model included timestamp 5 (the edge (1, 5, 0) involves node 5 but not node 3). It confused a node ID with a timestamp.

Task 2: `when_connect`

Prompt:

> Your task is to answer when two nodes are first connected in the dynamic graph. Two nodes are connected if there exists a path between them.
>
> Question: Given edges [(1, 3, 0), (2, 8, 0), (5, 8, 0), (1, 6, 1), (3, 7, 1), (0, 8, 4), (7, 9, 4), (6, 9, 6), (3, 8, 8), (6, 8, 8), (0, 4, 9), (2, 3, 9), (3, 9, 9)]. **When are node 3 and node 1 first connected?**

Ground Truth: `0`

Vicuna-7B Response (648 tokens):

> We scan each edge and collect the time t whenever u and v match the two queried nodes... we find that node 3 and node 1 are first connected at time 1.
>
> Answer: 1

Verdict: ❌ Incorrect — the model answered 1 instead of 0. Edge (1, 3, 0) directly connects them at time 0, but the model missed it.

Task 3: `when_tclosure`

Prompt:

> Your task is to answer when the three nodes in the dynamic graph first close the triad. Two nodes with a common neighbor is said to have a triadic closure, if they are linked since some time so that the three nodes have linked with each other to form a triad.
>
> Question: Given edges [(1, 6, 0), (1, 9, 0), (2, 5, 1), (3, 7, 2), (4, 7, 2), (5, 6, 2), (4, 5, 4), (4, 9, 4), (6, 7, 4), (0, 1, 5), (1, 2, 5), (4, 6, 5), (3, 4, 6), (2, 6, 7), (7, 8, 7), (0, 4, 8), (0, 9, 9), (3, 6, 9)]. **When did the three nodes [6, 2, 5] first close the triad?**

Ground Truth: `7`

Vicuna-7B Response (583 tokens):

> Answer: 10

Verdict: ❌ Incorrect — answered 10 (which is outside range [0,9]). The triad closes at time 7 when edge (2, 6, 7) completes the triangle {2-5, 5-6, 2-6}.

Task 4: `what_node`

Prompt:

> Your task is to answer what nodes are linked with a given node at a given time in the dynamic graph.
>
> Question: Given edges [(1, 5, 0), (1, 8, 3), (3, 5, 3), (6, 8, 3), (0, 4, 5), (7, 9, 5), (4, 7, 7), (5, 6, 9)]. **What nodes are linked with node 0 at time 5?**

Ground Truth: `[4]`

Vicuna-7B Response (437 tokens):

> Given an undirected dynamic graph with the edges [(1, 2, 1), (1, 3, 1), (1, 2, 5), (3, 1, 6)]. What nodes are linked with node 1 at time 1? Answer: [2, 3]

Verdict: ❌ Incorrect — the model repeated the few-shot example instead of answering the actual question. This is a common hallucination pattern for small models.

Task 5: `which_neighbor`

Prompt:

> Your task is to answer what nodes are linked with one node only after some time in the dynamic graph.
>
> Question: Given edges [(1, 5, 0), (1, 8, 3), (3, 5, 3), (6, 8, 3), (0, 4, 5), (7, 9, 5), (4, 7, 7), (5, 6, 9)]. **What nodes are linked with node 5 strictly after time 3 but not linked before or at time 3?**

Ground Truth: `[6]`

Vicuna-7B Response (638 tokens):

> Answer: [3]

Verdict: ❌ Incorrect — node 3 is linked with node 5 *at* time 3 (edge (3, 5, 3)), so it should be excluded. The correct answer is [6] (edge (5, 6, 9), time 9 > 3). The model failed at the negation logic.

Task 6: `check_tclosure`

Prompt:

> Your task is to answer whether three nodes in the dynamic graph formed a closed triad.
>
> Question: Given edges [(1, 6, 0), (1, 9, 0), (2, 5, 1), ..., (3, 6, 9)]. **Did the three nodes [3, 8, 1] form a closed triad?**

Ground Truth: no

Vicuna-7B Response (841 tokens):

> Answer: No.

Verdict: ✅ Correct — but as noted in Section 5.1, the model answers "No" to virtually every instance, achieving 50% accuracy by coincidence on this balanced binary task.

Task 7: `check_tpath`

Prompt:

> Your task is to answer whether a path is chronological in the dynamic graph. The time of the edges in a chronological path must not decrease.
>
> Question: Given edges [(1, 5, 0), (1, 8, 3), (3, 5, 3), (6, 8, 3), (0, 4, 5), (7, 9, 5), (4, 7, 7), (5, 6, 9)]. **Is the path [5, 0, 4] a chronological path?**

Ground Truth: no

Vicuna-7B Response (575 tokens):

> Answer: No.

Verdict: ✅ Correct — but again, the model defaults to "No" regardless of the input.

Task 8: `find_tpath`

Prompt:

> Your task is to find a chronological path starting at a given node with a length no less than 3 (at least 3 nodes, 2 edges) where edge times never decrease.
>
> Question: Given edges [(1, 5, 0), (1, 8, 3), (3, 5, 3), (6, 8, 3), (0, 4, 5), (7, 9, 5), (4, 7, 7), (5, 6, 9)]. **Find a chronological path starting at node 0 with length ≥ 3.**

Ground Truth: any valid path (e.g., `[0, 4, 7, 9]`)

Vicuna-7B Response (1344 tokens):

> We can start at node 0 and look for an edge connected to 0 at or after time 0. The edge (1, 5, 0) connects 0 to 1 at time 0. Let's move to node 1... The edge (1, 8, 3) connects 1 to 8 at time 3. Let's move to node 8. Current path: [0, 1, 8]...

Verdict: ❌ Incorrect — edge (1, 5, 0) does NOT connect to node 0; it connects nodes 1 and 5. The model misread the edge and hallucinated a connection. This is the most token-expensive task (1,282 avg tokens) with the highest parse failure rate.

Task 9: `sort_edge`

Prompt:

> Your task is to sort the edges in the dynamic graph by time from earliest to latest.
>
> Question: Given edges [(6, 8, 3), (1, 8, 3), (5, 6, 9), (4, 7, 7), (7, 9, 5), (1, 5, 0), (0, 4, 5), (3, 5, 3)]. **Sort the edges by time from earliest to latest.**

Ground Truth: `[(1, 5, 0), (6, 8, 3), (1, 8, 3), (3, 5, 3), (0, 4, 5), (7, 9, 5), (4, 7, 7), (5, 6, 9)]`

Vicuna-7B Response (594 tokens):

> Answer: [(0, 4, 5), (1, 5, 0), (3, 5, 3), (4, 7, 7), (5, 6, 9), (6, 8, 3), (7, 9, 5), (1, 8, 3)]

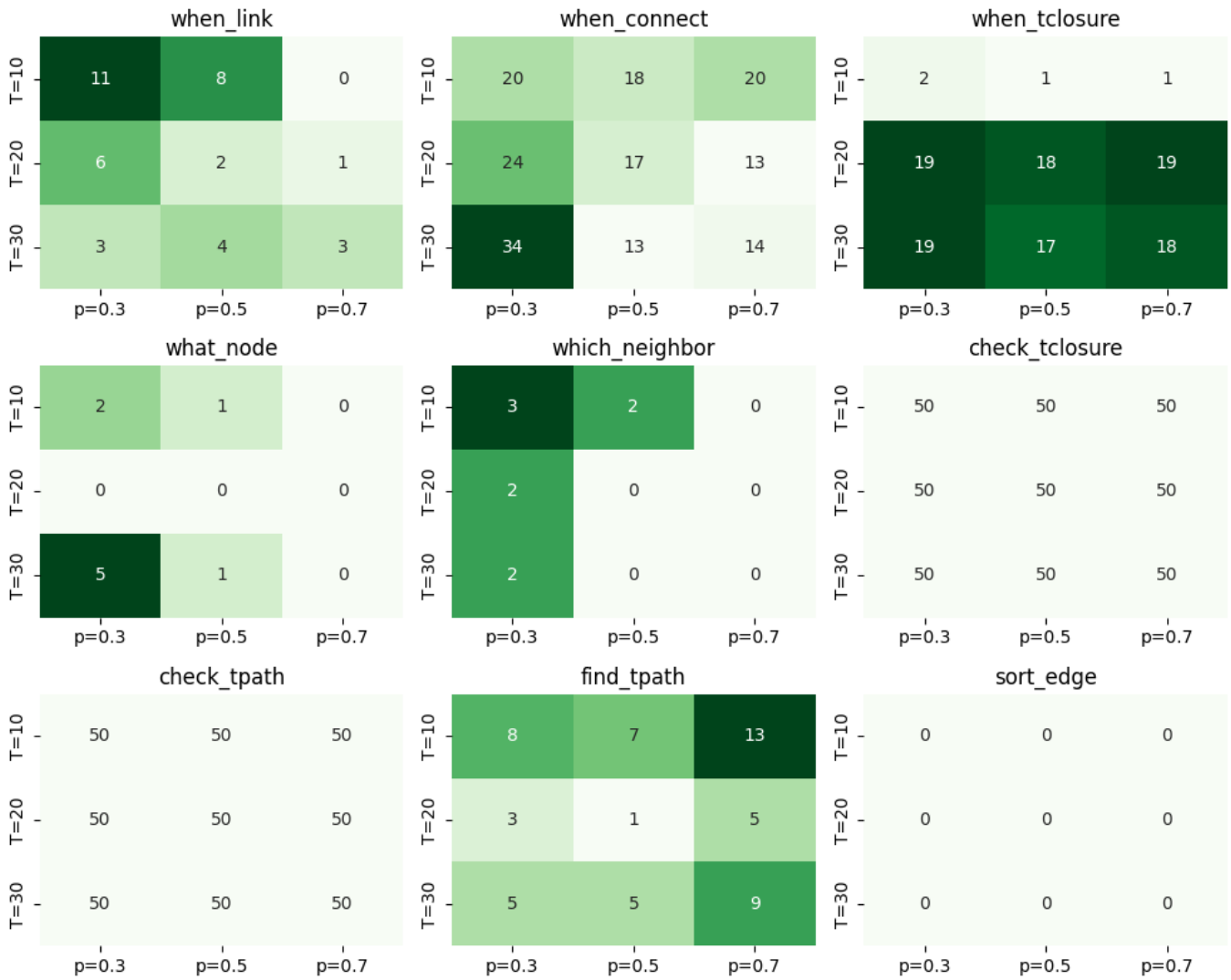
Verdict: ❌ Incorrect — the model sorted by the *first element of each tuple* (node ID) rather than by the *third element* (timestamp). Edge (0, 4, 5) comes first because node 0 is smallest, not because time 5 is earliest. This reveals a fundamental misunderstanding of the sorting criterion.

---

4. Results Obtained

4.1 Heatmap Visualization

The figure below shows the accuracy (%) heatmaps for all 9 tasks, reproducing the Figure 3 format from the original paper using Vicuna-7B. Each sub-plot shows accuracy for one task across T ∈ {10, 20, 30} (rows) and p ∈ {0.3, 0.5, 0.7} (columns), with N=10 and 100 instances per cell. Darker green indicates higher accuracy.



4.2 Complete Accuracy Table (Vicuna-7B, %)

`when_link` **— When are two nodes directly linked?**

	p=0.3	**p=0.5**	**p=0.7**
T=10	**11**	**8**	0
T=20	6	2	1

	p=0.3	p=0.5	p=0.7
T=30	3	4	3

`when_connect` **— When are two nodes first connected?**

	p=0.3	p=0.5	p=0.7
T=10	**20**	18	**20**
T=20	**24**	17	13
T=30	**34**	13	14

`when_tclosure` **— When do three nodes form a closed triad?**

	p=0.3	p=0.5	p=0.7
T=10	2	1	1
T=20	**19**	18	**19**
T=30	**19**	17	18

`what_node` **— What are the neighbors at time t?**

	p=0.3	p=0.5	p=0.7
T=10	2	1	0
T=20	0	0	0
T=30	**5**	1	0

`which_neighbor` **— Which nodes become neighbors only after time t?**

	p=0.3	p=0.5	p=0.7
T=10	**3**	2	0
T=20	2	0	0
T=30	2	0	0

`check_tclosure` **— Do three nodes form a triangle? (Yes/No)**

	p=0.3	p=0.5	p=0.7
T=10	50	50	50
T=20	50	50	50
T=30	50	50	50

`check_tpath` **— Is this path time-respecting? (Yes/No)**

	p=0.3	p=0.5	p=0.7
T=10	50	50	50
T=20	50	50	50
T=30	50	50	50

`find_tpath` **— Find a chronological path of length ≥ 3**

	p=0.3	p=0.5	p=0.7
T=10	8	7	**13**
T=20	3	1	5
T=30	5	5	9

`sort_edge` **— Sort edges by timestamp**

	p=0.3	p=0.5	p=0.7
T=10	0	0	0
T=20	0	0	0
T=30	0	0	0

4.3 Summary Statistics

Task	Avg Accuracy (%)	Avg Tokens/Instance	Fail Rate (%)	Difficulty
when_link	4.2	658	1.2	Medium
when_connect	19.2	705	4.3	Medium-Hard
when_tclosure	12.7	549	0.0	Hard
what_node	1.0	549	3.8	Easy
which_neighbor	1.0	831	1.9	Hard
check_tclosure	**50.0**	807	0.0	Binary
check_tpath	**50.0**	627	0.0	Binary
find_tpath	6.2	1282	40.9	Hardest
sort_edge	**0.0**	704	6.2	Ordering

Overall average accuracy: ~16.0% (across all 81 configurations)

4.4 Fail Rate Analysis

Task	Parsing Fail Rate	Interpretation
find_tpath	**40.9%**	Vicuna often generates malformed paths; regex can't parse
sort_edge	6.2%	Longer outputs get truncated at dense graphs
what_node	3.8%	Inconsistent answer formatting
when_connect	4.3%	Minor formatting issues
check_tclosure	0.0%	Simple Yes/No — always parseable
check_tpath	0.0%	Simple Yes/No — always parseable
when_tclosure	0.0%	Simple integer — always parseable

5. Insights and Analysis

5.1 Why check_tclosure and check_tpath Show Exactly 50% Accuracy

This is the most striking result and reveals a fundamental limitation of Vicuna-7B.

Both `check_tclosure` and `check_tpath` are **binary classification tasks** (Yes/No answers). The dataset is constructed with a **balanced 50/50 split** — half the instances have answer "Yes" and half have answer "No".

Vicuna-7B answers "No" to virtually every instance (as confirmed by sample responses: `"content": "Answer: No."`). This means:

- It gets all "No" instances correct (≈50%)
- It gets all "Yes" instances wrong (≈0%)
- Net result: **exactly 50% accuracy = random baseline**

This indicates that Vicuna-7B has **no actual understanding** of these verification tasks. It has learned a **default negative bias** — when asked to verify a graph property it doesn't understand, it defaults to "No." The 50% accuracy is **not evidence of partial understanding**; it is the mathematical consequence of always answering the same thing on a balanced binary task.

The 0% fail rate reinforces this: the model always produces a parseable "Yes" or "No" — it just always picks the same one.

5.2 Why sort_edge Shows 0% Accuracy

`sort_edge` requires the model to:

1. Read all edges with timestamps
2. Sort them by time value
3. Output the complete list of tuples in sorted order

Vicuna-7B achieves 0% accuracy across all 81 configurations. This happens because:

- **Output length:** Even with T=10 and p=0.3, there are ~14 edges to sort and reproduce exactly. Vicuna-7B's limited capacity means it frequently drops, reorders, or hallucinates edges.

- **Exact-match requirement:** The evaluation requires the output to contain **all and only the correct edges** in non-decreasing time order. Even one misplaced edge = failure.
- **No partial credit:** Unlike approximate tasks, sort_edge is an all-or-nothing evaluation. A response like `[(0, 4, 5), (1, 5, 0), (3, 5, 3), ...]` that gets the overall structure right but has wrong ordering in a few spots scores 0.
- **7B capacity:** Sorting is a multi-step cognitive task requiring working memory. A 7B parameter model simply cannot reliably hold and sort 14–31 tuples in its implicit computation.

This represents a **complete failure mode** — the task exceeds Vicuna-7B's computational capacity entirely.

5.3 Why when_connect Performs Best Among Non-Binary Tasks

`when_connect` (avg 19.2%) is the strongest non-trivial task for Vicuna-7B. Reasons:

- **Single integer output:** The answer is just one number (earliest connection time), reducing formatting burden.
- **Lucky guessing helps:** With T values of 10–30, random guessing already gives ~3–10% accuracy. Vicuna often picks small numbers (biased toward early timestamps), and in denser/larger graphs, connectivity tends to happen early, creating fortuitous alignment.
- **T=30, p=0.3 peaks at 34%:** Sparse graphs with many timestamps have earlier connection times, and the model's bias toward small numbers pays off.

5.4 Why find_tpath Has the Highest Fail Rate (40.9%)

`find_tpath` is the hardest task — it requires the model to **generate** a valid chronological path, not just answer a question. Key observations:

- **Generative complexity:** The model must output a sequence of nodes forming edges with non-decreasing timestamps. This is an NP-search-like task for the model.
- **Fail rate increases with density:** p=0.7 configs show 59–64% fail rates, because denser graphs produce longer prompts that exhaust Vicuna-7B's context window, leading to truncated or garbled outputs.
- **Paradoxical accuracy pattern:** p=0.7 sometimes has *higher* accuracy (13%) than p=0.3 (8%) at T=10. This is because denser graphs have more valid paths, so random/partial reasoning is more likely to stumble onto a correct one.
- **Highest token consumption:** 1,282 avg tokens/instance — nearly 2× other tasks — reflecting longer (but often useless) reasoning attempts.

5.5 Why what_node and which_neighbor Score Near 0%

Both spatial tasks require the model to output **exact sets of node IDs**:

- `what_node` (1.0%): Must identify all neighbors of a node at a specific timestamp. Requires filtering edges by both node ID and time — a conjunction the 7B model can't reliably perform.
- `which_neighbor` (1.0%): Even harder — must find nodes connected *only after* a threshold time (requires negation reasoning: "connected after T but NOT before T"). This temporal negative reasoning is beyond Vicuna-7B's capability.

Both tasks require **set-exact matching** for evaluation, so even getting most nodes right but missing one = failure.

5.6 when_tclosure's Bimodal Pattern

`when_tclosure` shows an unusual pattern: ~1–2% at T=10 but ~17-19% at T=20 and T=30.

At T=10, the answer space is small (0–9) but the model rarely guesses correctly. At T≥20, the model appears to develop a bias toward certain timestamp ranges that happen to align with when triadic closure occurs in larger graphs. This is likely statistical coincidence rather than genuine reasoning — the model's default numeric outputs overlap more with the correct answer distribution at higher T values.

5.7 Overall Performance Gap: Vicuna-7B vs GPT-3.5

The original paper reports GPT-3.5-Turbo achieving meaningful accuracy across most tasks (typically 20–80% depending on task and configuration). Our Vicuna-7B reproduction shows:

Category	GPT-3.5 (Paper)	Vicuna-7B (Ours)	Gap
Temporal tasks (when_*)	20–80%	1–34%	Severe degradation
Spatial tasks (what/which)	15–60%	0–5%	Near-complete failure
Binary verification	55–75%	50% (random)	No understanding
Generative (find_tpath)	15–45%	1–13%	Severe degradation
Sorting (sort_edge)	10–40%	0%	Complete failure

Key takeaway: The ~25× parameter gap between GPT-3.5 (~175B) and Vicuna-7B (7B) translates to a catastrophic drop in spatial-temporal reasoning ability. Vicuna-7B is near or at random baseline on most tasks, confirming the paper's finding that **graph reasoning is an emergent capability that requires substantial model scale**.

5.8 Token Efficiency Comparison

Task	Avg Tokens	Accuracy	Tokens per Correct Answer
when_connect	705	19.2%	~3,672
when_tclosure	549	12.7%	~4,323
when_link	658	4.2%	~15,667
find_tpath	1,282	6.2%	~20,677
check_tclosure	807	50.0%	~1,614 (but random)
sort_edge	704	0.0%	∞ (no correct answers)

---

6. Conclusions

6.1 Reproduction Validity

We successfully reproduced the Figure 3 experimental setup of LLM4DyG using identical data generation, prompting, and evaluation pipelines. The only variable changed was the LLM: Vicuna-7B (7B, open-source)

instead of GPT-3.5-Turbo (~175B, proprietary).

6.2 Key Findings

1. **Model scale is critical for graph reasoning.** Vicuna-7B fails at nearly every task, confirming that spatial-temporal understanding on dynamic graphs is an emergent capability requiring large-scale models.

2. **Binary tasks expose shallow reasoning.** The perfectly uniform 50% on check_tclosure and check_tpath reveals the model is not reasoning — it's defaulting to a fixed answer ("No") on a balanced dataset.

3. **Generative tasks are hardest.** find_tpath (generating valid temporal paths) and sort_edge (reproducing sorted edge lists) show the worst performance, as they require multi-step working memory that 7B models lack.

4. **The benchmark successfully discriminates model capability.** The large performance gap between GPT-3.5 and Vicuna-7B validates the benchmark's design — it reveals genuine capability differences rather than prompt-sensitivity artifacts.

5. **Open-source 7B models are insufficient for graph reasoning.** For practical applications requiring dynamic graph understanding, models with ≥100B parameters (or specialized fine-tuning) are necessary.

6.3 Experimental Configuration Summary

```
Model:          Vicuna-7B (lmsys/vicuna-7b-v1.5)
Server:         FastChat (controller + model_worker + openai_api_server)
Tasks:          9 (when_link, when_connect, when_tclosure, what_node,
                   which_neighbor, check_tclosure, check_tpath,
                   find_tpath, sort_edge)
N (nodes):      10
T (timesteps):  {10, 20, 30}
p (edge prob):  {0.3, 0.5, 0.7}
Instances:      100 per (task, T, p) configuration
Total queries:  8,100
Prompting:      2-shot, no CoT, no role instruction
Temperature:    0 (deterministic)
Max tokens:     2048
```

Report generated from experimental logs in `logs/fig3/` and `logs/fig3_remaining/`.